STAFF ENGINEER DEEP DIVE · LLD SERIES

From OCP to Strategy Pattern: The Problem Nobody Talks About

A story-driven technical journey through real engineering pain → OCP → Strategy Pattern. Because patterns are not invented. They are forced into existence by real problems.

Chapter 1 — The Business Story That Started It All

It's 2019. You're a backend engineer at **ShopEasy**, a growing e-commerce startup. Your PM walks over and says:

> **SPRINT TICKET**
>
> "We need to support Card payments at checkout. Keep it simple, we're launching in 2 weeks."

You write what every engineer writes in week one: clean, focused, simple code.

Version 1 — Card Only

PaymentService.java

```java
public class PaymentService {

    public void processPayment(String type, double amount) {
        // Only card in V1. Straightforward.
        if (type.equals("CARD")) {
            System.out.println("Processing CARD payment of ₹" + amount);
            // card gateway API call...
        }
    }
}
```

payment_service.py

```python
class PaymentService:
    def process_payment(self, type: str, amount: float):
        if type == "CARD":
            print(f"Processing CARD payment of ₹{amount}")
            # card gateway call...
```

This looks completely fine. It passes code review. It ships. 🎉

3 MONTHS LATER

Version 1 — Card → Version 2 — + UPI

The PM is back: "UPI is huge in India. We need to add it ASAP."

You open the file. "Easy," you think. You add an `else if`.

PaymentService.java — Version 2

```java
public class PaymentService {

    public void processPayment(String type, double amount) {
        if (type.equals("CARD")) {
            System.out.println("Processing CARD payment of ₹" + amount);
        } else if (type.equals("UPI")) {               // ← new
            System.out.println("Processing UPI payment of ₹" + amount);
        }
    }
}
```

Still fine. Still readable. You ship it again. ✅

2 MONTHS LATER

Card → UPI → + Wallet

Now it's Wallet support. Then two months after that — Crypto. Your `PaymentService` now looks like this:

PaymentService.java — Version 4 (The Painful Reality)

```java
public class PaymentService {

    public void processPayment(String type, double amount) {

        if (type.equals("CARD")) {
            // validate card number
            // check card expiry
            // call Stripe/Razorpay gateway
            // handle 3D-secure auth
            // retry on timeout
            System.out.println("Card payment processed: ₹" + amount);

        } else if (type.equals("UPI")) {
            // validate UPI ID format
            // initiate collect request
            // poll NPCI for status
            // handle VPA errors
            System.out.println("UPI payment processed: ₹" + amount);

        } else if (type.equals("WALLET")) {
            // check wallet balance
            // deduct from wallet ledger
            // handle insufficient funds
            // update loyalty points
            System.out.println("Wallet payment processed: ₹" + amount);

        } else if (type.equals("CRYPTO")) {
            // connect to blockchain node
            // calculate gas fees
            // sign transaction
            // wait for block confirmation
            System.out.println("Crypto payment processed: ₹" + amount);
        }
    }
}
```

> 🔥 **THE PAIN POINTS — LET'S NAME THEM PRECISELY**
>
> This file has grown to 200+ lines. It now carries 4 completely different responsibilities. A bug fix in CARD logic requires redeploying ALL payment logic. A junior engineer accidentally breaks UPI while fixing WALLET. Tests for Crypto logic accidentally fail Card tests. The PM asks for a 5th method: "Net Banking". You open the file and feel dread.

Why This Code Is Genuinely Dangerous in Production

Risk 1 — Regression: Every time you touch this class, you risk breaking code that was working fine. The CARD flow was battle-tested for 6 months. The moment you add Crypto, you must retest Card, UPI, and Wallet too — not because they changed, but because they live in the same class.

Risk 2 — Deployment blast radius: In a microservices world or even a monolith with CI/CD, any change to this file triggers a full redeploy. A 1-line fix for a Crypto bug forces a deployment that touches the Card flow — which processes ₹10 crore daily.

Risk 3 — Growing complexity over time: Payment methods have unique retry logic, timeout handling, error codes, audit trails. Each branch will grow independently. You'll have a 2000-line function in 18 months.

Risk 4 — Testing hell: You cannot write a unit test for "just UPI" — you must import the entire `PaymentService`. Every test file is now coupled to all payment types.

THE QUESTION THAT FORCED OCP INTO EXISTENCE

Chapter 2 — What Exact Engineering Pain Forced OCP?

> **THE CORE QUESTION**
>
> "Why should adding a new payment method require touching code that handles old payment methods?"
>
> Answer: It shouldn't. That's exactly the pain OCP was designed to eliminate.

Robert C. Martin's **Open/Closed Principle** states: "A software entity should be open for extension but closed for modification."

But forget the definition for now. Here's what it means in our story:

> *"When a new payment method arrives, I should be able to add it without touching any existing, working, tested, deployed code."*

How Do We Achieve That? — Abstraction

The insight is: every payment method does the same thing — it processes a payment. They differ in how they do it. So we extract the "what" into an interface, and push the "how" into separate classes.

```
BEFORE OCP (Monolithic if-else): ══════════════════════════
┌──────────────────────────────────────┐ │ PaymentService │ │ │ │ processPayment(type, amount) { │
│ if CARD → card logic (50L) │ │ if UPI → upi logic (40L) │ │ if WALLET → wallet logic (45L) │ │ if
CRYPTO → crypto logic (60L) │ │ } │ │ │ │ ⚠ All 4 methods in ONE class │ │ ⚠ Change any → redeploy all
│ │ ⚠ Bug in one → fear for others │ └──────────────────────────────────────┘ AFTER OCP
(Abstraction + Extension): ══════════════════════════════ «interface» ┌──────────────────┐ │
Payment │ │ ──────────────── │ │ pay(amount) │ └──────────┬──────────┘ │ implements
┌──────┼──────────────────┐ ▼ ▼ ▼ ▼ ┌──────┐ ┌──────┐ ┌──────────┐ ┌──────────┐ │
Card │ │ UPI │ │ Wallet │ │ Crypto │ │Payment │ │Pay't │ │Payment │ │Payment │ └──────┘ └──────┘
└──────┘ └──────┘ ✅ New method = new class only ✅ Existing classes NEVER touched ✅ Deploy only
the new class ✅ Test only the new class
```

OCP Implementation — Java

Payment.java — The Interface (The Contract)

```java
// This is the abstraction. It never changes.
// Every payment method MUST honour this contract.
public interface Payment {
    void pay(double amount);
}
```

CardPayment.java

```java
public class CardPayment implements Payment {

    @Override
    public void pay(double amount) {
        System.out.println("[CARD] Validating card details...");
        System.out.println("[CARD] Calling Razorpay gateway...");
        System.out.println("[CARD] Payment of ₹" + amount + " successful.");
    }
}
```

UpiPayment.java

```java
public class UpiPayment implements Payment {

    @Override
    public void pay(double amount) {
        System.out.println("[UPI] Initiating collect request...");
        System.out.println("[UPI] Polling NPCI for status...");
        System.out.println("[UPI] Payment of ₹" + amount + " successful.");
    }
}
```

WalletPayment.java + CryptoPayment.java

```java
public class WalletPayment implements Payment {
    @Override
    public void pay(double amount) {
        System.out.println("[WALLET] Checking balance...");
        System.out.println("[WALLET] Debiting ₹" + amount + " from wallet.");
    }
}

public class CryptoPayment implements Payment {
    @Override
    public void pay(double amount) {
        System.out.println("[CRYPTO] Signing blockchain transaction...");
        System.out.println("[CRYPTO] Waiting for block confirmation...");
        System.out.println("[CRYPTO] Payment of ₹" + amount + " confirmed.");
    }
}
```

Now adding **Net Banking**? You write one new class. You **never open any existing file**. That's OCP in action.

OCP Implementation — Python

payments.py

```python
from abc import ABC, abstractmethod

class Payment(ABC):
    @abstractmethod
    def pay(self, amount: float) -> None:
        pass


class CardPayment(Payment):
    def pay(self, amount: float) -> None:
        print(f"[CARD] Processing ₹{amount} via Razorpay...")

class UpiPayment(Payment):
    def pay(self, amount: float) -> None:
        print(f"[UPI] Processing ₹{amount} via NPCI...")

class WalletPayment(Payment):
    def pay(self, amount: float) -> None:
        print(f"[WALLET] Debiting ₹{amount} from wallet...")

class CryptoPayment(Payment):
    def pay(self, amount: float) -> None:
        print(f"[CRYPTO] Confirming blockchain transaction of ₹{amount}...")
```

> ✅ **WHAT OCP SOLVED**
>
> **Modification risk:** Adding Crypto never touches CardPayment.
> **Regression risk:** You deploy only CryptoPayment.class. Card, UPI, Wallet are untouched.
> **Testability:** Unit test CardPayment in complete isolation — no other class is involved.
> **Extensibility:** New payment = new file. Open for extension, closed for modification.

BUT WAIT...

Chapter 3 — Did OCP Solve Everything?

> ⚠ **THE PROBLEM THAT REMAINED**
>
> OCP told us HOW to write each payment method in isolation. It did NOT tell us WHO decides which method to use at runtime.

Watch this. Even after applying OCP perfectly, someone still has to select the right implementation. Let's look at what your controller or service layer looks like:

CheckoutController.java — The Problem Hiding in Plain Sight

```java
public class CheckoutController {

    public void checkout(String paymentType, double amount) {

        Payment payment;

        // 🚨 The if-else monster is back. Just in a different place.
        if (paymentType.equals("CARD")) {
            payment = new CardPayment();
        } else if (paymentType.equals("UPI")) {
            payment = new UpiPayment();
        } else if (paymentType.equals("WALLET")) {
            payment = new WalletPayment();
        } else if (paymentType.equals("CRYPTO")) {
            payment = new CryptoPayment();
        } else {
            throw new IllegalArgumentException("Unknown payment type");
        }

        payment.fn(amount);
    }
}
```

> 🔥 **NAME THE REMAINING PROBLEMS PRECISELY**
>
> **Problem 1 — Object creation responsibility:** CheckoutController now violates SRP. Its job is to handle checkout flow — not to decide which payment class to instantiate.
>
> **Problem 2 — Selection logic grows forever:** Every new payment method adds another `else if` here. We moved the problem, we didn't solve it.
>
> **Problem 3 — Tight coupling with concrete classes:** CheckoutController directly references `new CardPayment()`, `new UpiPayment()` etc. It knows about every concrete implementation.
>
> **Problem 4 — Runtime inflexibility:** What if a user's preferred payment method changes mid-session? Or you want A/B test 2 payment gateways at runtime? The controller can't swap behavior on the fly.
>
> **Problem 5 — Testing the controller:** To unit test CheckoutController, you're forced to instantiate real CardPayment, UpiPayment etc. You can't easily mock them because the controller creates them internally.

```
THE REAL ARCHITECTURE PROBLEM: CheckoutController "knows too much"
────────────────────────────────── ┌──────────────────────────────┐ │
CheckoutController │ │ │ │ knows about CardPayment ✗ │ │ knows about UpiPayment ✗ │ │ knows about
WalletPayment ✗ │ │ knows about CryptoPayment ✗ │ │ │ │ creates objects itself ✗ │ │ selection logic
grows here ✗ │ └──────────────────────────────┘ This is called TIGHT COUPLING. OCP fixed the
implementation side. Something needs to fix the selection + coupling side.
```

THE EVOLUTIONARY NEXT STEP

Chapter 4 — Why Strategy Pattern Emerged Naturally

Strategy Pattern was not invented by someone reading a textbook. It was discovered by engineers who kept running into this exact problem: "I have a family of algorithms/behaviours. I need to select one at runtime, without the caller knowing which one was chosen, and without modifying the caller when new ones are added."

Big if-else — All logic in one class → **OCP Applied** — Separate implementation classes → **Remaining Problem** — if-else moved to caller → **Strategy Pattern** — Inject behaviour at runtime

The key insight of Strategy Pattern is this:

> *"Instead of the Context selecting and creating the strategy — let someone outside inject it. The Context should just execute whatever strategy it was given."*

The Three Moving Parts

```
STRATEGY PATTERN — FULL ARCHITECTURE: ══════════════════════════════ «interface»
┌──────────────┐ │ PaymentStrategy │ ← Strategy Interface │ ──────────────── │ (the contract) │
execute(amount)│ └──────────────────┘ │ implements ┌──────────┼──────────────────┐ ▼ ▼ ▼ ▼
┌──────┐ ┌────┐ ┌──────┐ ┌──────┐ │ Card │ │ UPI │ │ Wallet │ │ Crypto │ ← Concrete
│Strategy│ │Strat.│ │Strategy│ │Strategy│ Strategies └──────┘ └────┘ └──────┘ └──────┘ ▲
▲ ▲ ▲ └──────────────────────────┘ │ injected into │ ┌──────────▼──────────┐ │
PaymentContext │ ← Context │ ──────────────── │ (just executes) │ strategy: Payment │ │
executePayment() │ └──────────────────────┘ ▲ │ injects strategy ┌──────────┴──────────┐ │
CheckoutService │ ← Caller │ (knows which │ (or Factory/ │ strategy to use) │ DI Container)
└──────────────────┘ CALL FLOW: ══════════ 1. User selects "UPI" at checkout 2. CheckoutService
creates UpiPaymentStrategy 3. CheckoutService injects it into PaymentContext 4.
PaymentContext.executePayment(500) is called 5. Context delegates → UpiPaymentStrategy.execute(500) 6.
UpiStrategy handles all UPI-specific logic 7. Context never knew it was UPI
```

Complete Java Implementation

PaymentStrategy.java — Strategy Interface

```java
/**
 * Strategy Interface.
 * Every payment strategy must implement this.
 * This is the ONLY thing Context will ever know about.
 */
public interface PaymentStrategy {
    void execute(double amount);
}
```

Concrete Strategies — CardPaymentStrategy.java

```java
public class CardPaymentStrategy implements PaymentStrategy {

    private final String cardNumber;
    private final String cvv;

    public CardPaymentStrategy(String cardNumber, String cvv) {
        this.cardNumber = cardNumber;
        this.cvv = cvv;
    }

    @Override
    public void execute(double amount) {
        System.out.println("[CARD] Validating card: " + mask(cardNumber));
        System.out.println("[CARD] Charging ₹" + amount + " via Razorpay...");
        System.out.println("[CARD] ✅ Payment successful.");
    }

    private String mask(String card) {
        return "****-****-****-" + card.substring(card.length() - 4);
    }
}


public class UpiPaymentStrategy implements PaymentStrategy {

    private final String upiId;

    public UpiPaymentStrategy(String upiId) {
        this.upiId = upiId;
    }

    @Override
    public void execute(double amount) {
        System.out.println("[UPI] VPA: " + upiId);
        System.out.println("[UPI] Collecting ₹" + amount + " via NPCI...");
        System.out.println("[UPI] ✅ Payment successful.");
    }
}


public class WalletPaymentStrategy implements PaymentStrategy {

    private final String walletId;

    public WalletPaymentStrategy(String walletId) {
        this.walletId = walletId;
    }

    @Override
    public void execute(double amount) {
        System.out.println("[WALLET] Wallet ID: " + walletId);
        System.out.println("[WALLET] Debiting ₹" + amount + "...");
        System.out.println("[WALLET] ✅ Payment successful.");
    }
}


public class CryptoPaymentStrategy implements PaymentStrategy {

    private final String walletAddress;

    public CryptoPaymentStrategy(String walletAddress) {
        this.walletAddress = walletAddress;
    }

    @Override
    public void execute(double amount) {
        System.out.println("[CRYPTO] Wallet: " + walletAddress);
        System.out.println("[CRYPTO] Signing transaction for ₹" + amount);
        System.out.println("[CRYPTO] Waiting for block confirmation...");
        System.out.println("[CRYPTO] ✅ Transaction confirmed.");
    }
}
```

PaymentContext.java — The Context

```java
/**
 * Context class.
 *
 * KEY DESIGN DECISIONS:
 * 1. It holds a reference to the Strategy Interface — NOT a concrete class.
 * 2. It does NOT decide which strategy to use. That's injected from outside.
 * 3. It can swap strategies at runtime (setStrategy).
 * 4. It delegates execution entirely to the strategy.
 *
 * PaymentContext has ZERO if-else blocks.
 * It doesn't know if it's processing Card, UPI, Wallet, or Crypto.
 * It just executes whatever strategy it holds.
 */
public class PaymentContext {

    private PaymentStrategy strategy;   // ← holds interface, not concrete class

    // Constructor injection — strategy provided at creation
    public PaymentContext(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    // Runtime swap — change strategy without recreating context
    public void setStrategy(PaymentStrategy strategy) {
        this.strategy = strategy;
    }

    // Execute — delegate completely. Zero logic here.
    public void executePayment(double amount) {
        this.strategy.execute(amount);
    }
}
```

CheckoutService.java — The Caller (Clean Runtime Selection)

```java
public class CheckoutService {

    public void checkout(String paymentType, String paymentDetail, double amount) {

        // Selection logic is isolated here, in one place.
        // Even better: this can be moved to a Factory or DI container.
        PaymentStrategy strategy = resolveStrategy(paymentType, paymentDetail);

        // Context doesn't know which strategy it's using.
        PaymentContext context = new PaymentContext(strategy);
        context.executePayment(amount);
    }

    private PaymentStrategy resolveStrategy(String type, String detail) {
        return switch (type.toUpperCase()) {
            case "CARD"   -> new CardPaymentStrategy(detail, "***");
            case "UPI"    -> new UpiPaymentStrategy(detail);
            case "WALLET" -> new WalletPaymentStrategy(detail);
            case "CRYPTO" -> new CryptoPaymentStrategy(detail);
            default      -> throw new IllegalArgumentException(
                                "Unsupported payment type: " + type);
        };
    }
}


// ─── Usage ──────────────────────────────────────────────
public class Main {
    public static void main(String[] args) {

        CheckoutService checkout = new CheckoutService();

        checkout.checkout("CARD",   "4111111111111234", 1500.0);
        checkout.checkout("UPI",    "ram@okaxis",        500.0);
        checkout.checkout("WALLET", "wallet_abc123",    250.0);
        checkout.checkout("CRYPTO", "0xABCDEF123456",   9999.0);

        // Runtime swap — same context, different strategy
        PaymentContext ctx = new PaymentContext(new CardPaymentStrategy("4111111111111234", "123"));
        ctx.executePayment(1000.0);

        // User switched to UPI mid-session? No problem.
        ctx.setStrategy(new UpiPaymentStrategy("ram@okaxis"));
        ctx.executePayment(1000.0);
    }
}
```

Complete Python Implementation

strategy_pattern.py — Full Implementation

```python
from abc import ABC, abstractmethod
from typing import Optional


# ── Strategy Interface ──────────────────────────────────
class PaymentStrategy(ABC):
    @abstractmethod
    def execute(self, amount: float) -> None:
        pass


# ── Concrete Strategies ─────────────────────────────────
class CardPaymentStrategy(PaymentStrategy):
    def __init__(self, card_number: str, cvv: str):
        self.card_number = card_number
        self.cvv = cvv

    def execute(self, amount: float) -> None:
        masked = f"****-****-****-{self.card_number[-4:]}"
        print(f"[CARD] Card: {masked}")
        print(f"[CARD] Charging ₹{amount} via Razorpay...")
        print("[CARD] ✅ Payment successful.\n")


class UpiPaymentStrategy(PaymentStrategy):
    def __init__(self, upi_id: str):
        self.upi_id = upi_id

    def execute(self, amount: float) -> None:
        print(f"[UPI] VPA: {self.upi_id}")
        print(f"[UPI] Collecting ₹{amount} via NPCI...")
        print("[UPI] ✅ Payment successful.\n")


class WalletPaymentStrategy(PaymentStrategy):
    def __init__(self, wallet_id: str):
        self.wallet_id = wallet_id

    def execute(self, amount: float) -> None:
        print(f"[WALLET] ID: {self.wallet_id}")
        print(f"[WALLET] Debiting ₹{amount}...")
        print("[WALLET] ✅ Payment successful.\n")


class CryptoPaymentStrategy(PaymentStrategy):
    def __init__(self, wallet_address: str):
        self.wallet_address = wallet_address

    def execute(self, amount: float) -> None:
        print(f"[CRYPTO] Wallet: {self.wallet_address}")
        print(f"[CRYPTO] Signing tx for ₹{amount}...")
        print("[CRYPTO] ✅ Transaction confirmed.\n")


# ── Context ─────────────────────────────────────────────
class PaymentContext:
    """
    Knows NOTHING about concrete payment types.
    Holds a reference to an interface.
    Executes whatever strategy it holds.
    Strategies can be swapped at runtime.
    """
    def __init__(self, strategy: PaymentStrategy):
        self._strategy = strategy

    def set_strategy(self, strategy: PaymentStrategy) -> None:
        self._strategy = strategy

    def execute_payment(self, amount: float) -> None:
        self._strategy.execute(amount)   # pure delegation


# ── Caller / Service ────────────────────────────────────
class CheckoutService:
    STRATEGY_MAP = {
        "CARD":   lambda detail: CardPaymentStrategy(detail, "***"),
        "UPI":    lambda detail: UpiPaymentStrategy(detail),
        "WALLET": lambda detail: WalletPaymentStrategy(detail),
        "CRYPTO": lambda detail: CryptoPaymentStrategy(detail),
    }

    def checkout(self, payment_type: str, detail: str, amount: float) -> None:
        factory = self.STRATEGY_MAP.get(payment_type.upper())
        if not factory:
            raise ValueError(f"Unsupported payment type: {payment_type}")

        strategy = factory(detail)
        context  = PaymentContext(strategy)
        context.execute_payment(amount)


# ── Usage ───────────────────────────────────────────────
if __name__ == "__main__":
    service = CheckoutService()

    service.checkout("CARD",   "4111111111111234", 1500.0)
    service.checkout("UPI",    "ram@okaxis",        500.0)
    service.checkout("WALLET", "wallet_abc123",     250.0)
    service.checkout("CRYPTO", "0xABCDEF123456",    9999.0)

    # Runtime strategy swap:
    ctx = PaymentContext(CardPaymentStrategy("4111111111111234", "123"))
    ctx.execute_payment(1000)

    ctx.set_strategy(UpiPaymentStrategy("ram@okaxis"))
    ctx.execute_payment(1000)   # same context, new behaviour
```

THE COMPARISON

Chapter 5 — Side-by-Side Comparison

Aspect	No OCP (if-else)	OCP Applied	Strategy Pattern
Extensibility	❌ Modify existing class for every new type	✅ Add new class, never touch old ones	✅ Add new strategy class — zero existing change
Coupling	❌ Caller tightly coupled to all implementations	⚠ Caller still creates concrete objects	✅ Caller depends only on Strategy interface
Testability	❌ Cannot test one method without importing all	⚠ Classes are testable; caller is still messy	✅ Inject mock strategy — test context in isolation
Runtime Flexibility	❌ Behaviour locked at compile time	❌ Behaviour still decided in caller's if-else	✅ setStrategy() swaps behaviour at runtime
Maintainability	❌ One class grows endlessly	⚠ Classes are small; selection logic sprawls	✅ Each concern in its own class; selection isolated
SRP Compliance	❌ One class = all responsibilities	⚠ Implementations are clean; caller violates SRP	✅ Context = execute. Strategy = behaviour. Clean.
Deployment Risk	❌ Every change deploys everything	⚠ New class = isolated deploy, but caller changes	✅ New strategy deploys alone, caller unchanged
Object Creation	❌ Buried inside the same class	❌ Caller does new CardPayment(), new UpiPayment()...	✅ Isolated in factory / DI / service layer

THE FULL SUMMARY

Chapter 6 — The Precise Answers

> ✅ **WHAT EXACT PROBLEM DID OCP SOLVE?**
>
> Every time a new payment method arrived, engineers had to **open the existing PaymentService class** and modify it. This created regression risk — working, tested, deployed code was disturbed unnecessarily. OCP said: **abstract the behaviour into an interface, push each implementation into its own class**. New method = new class. Existing classes are never touched, never retested, never redeployed.

> ⚠ **WHAT PROBLEM WAS STILL LEFT AFTER OCP?**
>
> OCP fixed the implementation side. But **the caller still had to select which implementation to use**. The `if-else` tree migrated from PaymentService to CheckoutController. The caller was still tightly coupled to every concrete class via `new CardPayment()`, `new UpiPayment()`. You could not swap strategies at runtime. You could not inject a mock for testing. The object creation and selection responsibility had no home.

> 💡 **WHY STRATEGY PATTERN WAS THE LOGICAL NEXT STEP**
>
> Strategy Pattern gave the remaining problem a name and a solution. **Extract the selection concern**. Let the Context depend only on an interface. Inject the concrete strategy from outside. This achieved: decoupled caller, runtime behaviour swapping, mockable strategies for testing, and a single, findable place for selection logic (factory or DI container). It completed what OCP started.

🎯 Interview Section — What Seniors Actually Ask

Trick Questions You'll Face

- **"If I apply OCP, do I still need Strategy Pattern?"**
  Answer: OCP tells you to separate implementations. Strategy tells you how to wire them at runtime without the caller knowing which one it's using. OCP = what to separate. Strategy = how to inject and switch.
- **"Is Strategy Pattern just Polymorphism with a fancy name?"**
  Answer: Polymorphism is the mechanism. Strategy is the design intent — the explicit decision that behaviour should be encapsulated, injected, and swappable at runtime. The pattern adds: Context class, Dependency Injection, and runtime swap semantics.
- **"Where exactly should the selection logic live?"**
  Answer: In production: a Factory class, a Strategy Registry (Map of type → strategy), or your DI container (Spring, Guice). Never in the Context. Never in the business service.
- **"How would you handle adding a 5th payment method in production with zero downtime?"**
  Answer: With Strategy Pattern + a registry map: add a new strategy class, register it in the map, deploy. Context and CheckoutService are untouched. Feature flag it. Zero downtime.

Common Mistakes Junior Engineers Make

- Putting selection logic inside the Context — defeats the entire pattern.
- Using Strategy Pattern for single-implementation scenarios — it's overkill without extensibility needs.
- Confusing Strategy with State Pattern — Strategy swaps behaviours, State swaps object state transitions.
- Not using Dependency Injection — hardcoding `new CardPaymentStrategy()` inside Context re-introduces coupling.

Real Industry Examples

- **Java's Comparator:** `Collections.sort(list, comparator)` — the comparator IS the strategy. You inject sorting behaviour at runtime.
- **Spring Security:** Authentication strategies (JWT, Session, OAuth2) are plugged in as strategies.
- **Kafka Serializers:** You inject a Serializer strategy — JSON, Avro, Protobuf — at runtime.
- **Tax Calculation Engines:** Different tax strategies per country, injected per user's geography.

Senior Engineer Perspective

- The real value of Strategy Pattern isn't the diagram — it's that your **business logic never touches infrastructure code**. PaymentContext has no idea it's processing Crypto.
- In DDD (Domain-Driven Design), strategies map naturally to domain services. Each payment strategy IS a domain concept with its own invariants.
- In a microservices architecture, each strategy can evolve into its own service — the interface becomes an API contract.

CLOSING THOUGHT

Design patterns are not rules invented in a textbook. They are crystallised solutions to recurring engineering pain. Every senior engineer who truly understands Strategy Pattern can trace its necessity backward: from a big if-else that grew too painful → to OCP that fixed the modification problem → to the realisation that something still had to decide and inject the right implementation → to Strategy Pattern, which gave that responsibility a clean, testable, swappable home.

Next time you see a growing `if-else` on types, you're looking at the exact moment where OCP and Strategy Pattern want to be born.

Day 5 · System Design Journey · LLD Series

#SystemDesign #LLD #DesignPatterns #StrategyPattern #OCP #SOLID #Java #Python #SoftwareEngineering #LearningInPublic